

EECS 836: Machine Learning

Zijun Yao

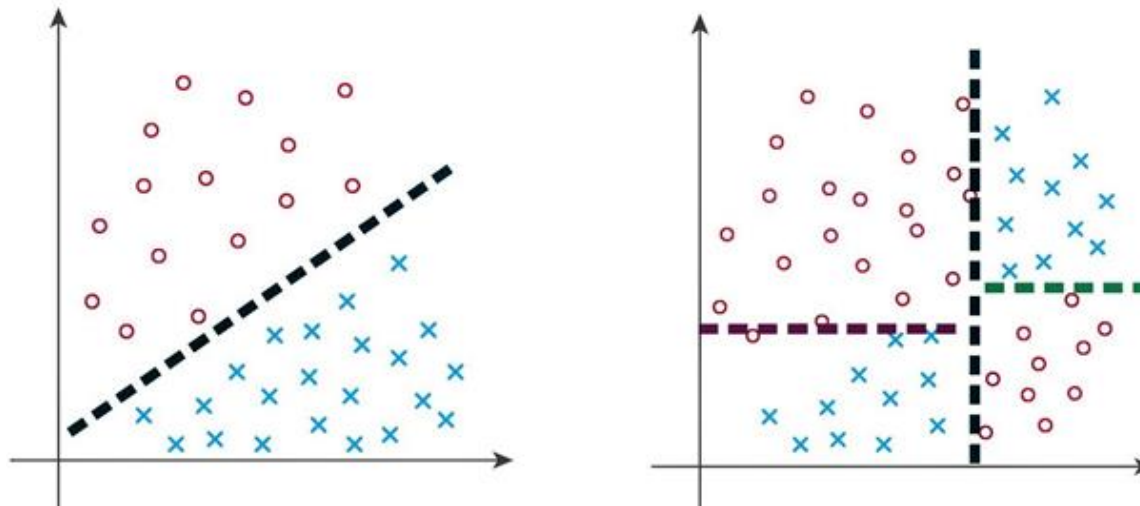
Assistant Professor, EECS Department
The University of Kansas

Agenda

- Ensemble Methods and Boosting
 - Voting, Bagging, Boosting
 - AdaBoost Algorithm

Weak learners

- Simple classifier that do only slightly better than random chance (50%)
- Like naïve Bayes, logistic regression, shallow decision tree

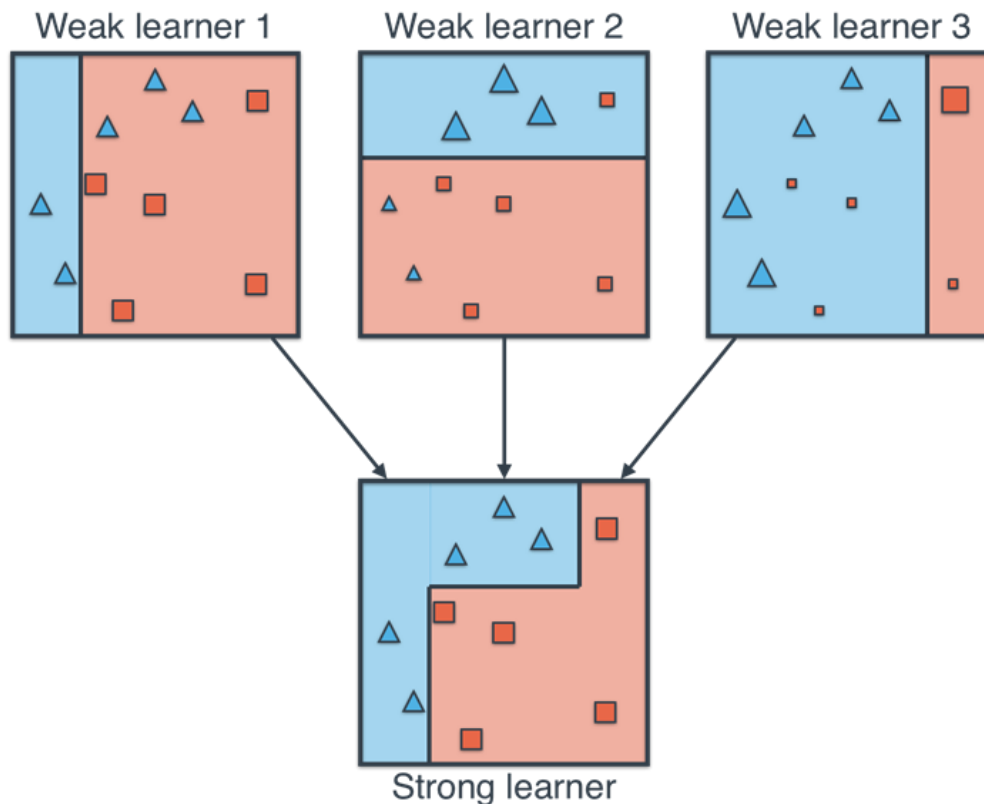


Good – Low variance, don't usually overfit (Simple “rule of thumb” are often correct)

Bad – High bias, can't solve hard learning problems (Can't find a single highly accurate prediction rule)

Ensemble methods

- Instead of learning a single (weak) classifier, learn **many weak classifiers that are good at different parts of the input space**
- Predict class label of test records by combining the predictions made by multiple classifiers (e.g., by taking majority vote)



We get a strong learner by assembling a set of weak learners – avengers of models!

Voting

- Voting – decide the output class by democracy:
 - Conclude the “opinion” of all classifiers
 - **Hard voting**: count the number of classifiers votes for “+” vs. “-”
 - **Soft voting**: classifiers that are most “sure” will vote with more conviction. Sum up the probabilities for “+” vs. “-” over all classifiers
 - **On average, do better than single classifier!**

Tests sample/ Classifier	y_prediction				y_test
	Logistic Regresssion	Decision Tree	SVM	Voting Classifier	
1. test sample	1	1	0	1	1
2. test sample	1	0	1	1	1
3. test sample	0	0	1	0	0
4. test sample	1	0	1	1	1
5. test sample	0	0	1	0	0
6. test sample	0	0	0	0	0
7. test sample	0	0	0	0	0
8. test sample	0	1	1	1	1
9. test sample	0	1	1	1	0
10. test sample	0	1	1	1	1
Accuracy	80%	70%	60%	90%	

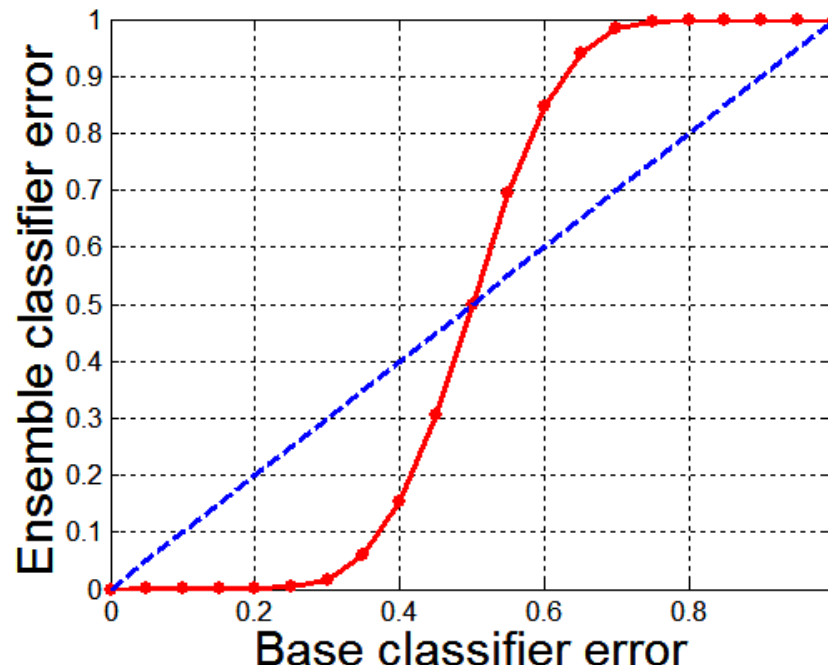
Why do ensemble methods work

- Suppose there are 25 weak classifiers
 - Each classifier has error rate $\epsilon = 0.35$
 - Hard voting is used by counting majority
 - If all classifiers are independent trained
 - Errors are uncorrelated - each classifier has error on different data points
- Error rate of ensemble = probability of having more than half of classifiers being wrong

$$e_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1 - \epsilon)^{25-i} = 0.06$$

Necessary conditions for ensemble methods

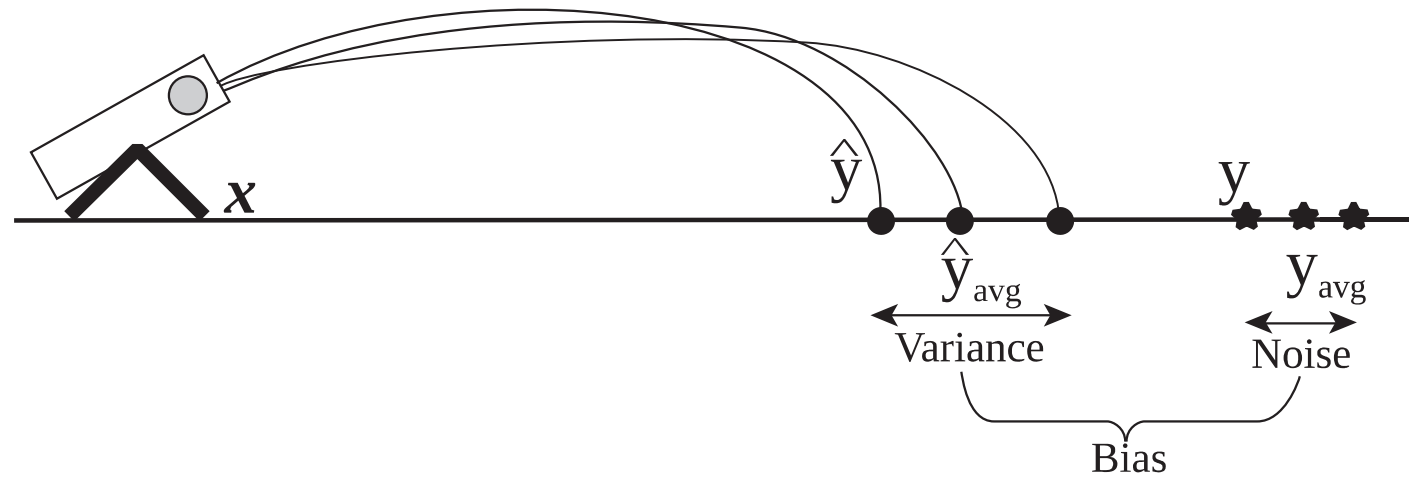
- Ensemble Methods work better than a single base classifier if:
 1. All base classifiers are independent of each other
 2. All base classifiers perform better than random guessing (error rate < 0.5 for binary classification)



Classification error for an ensemble of 25 base classifiers, assuming their errors are uncorrelated.

Bias-Variance decomposition

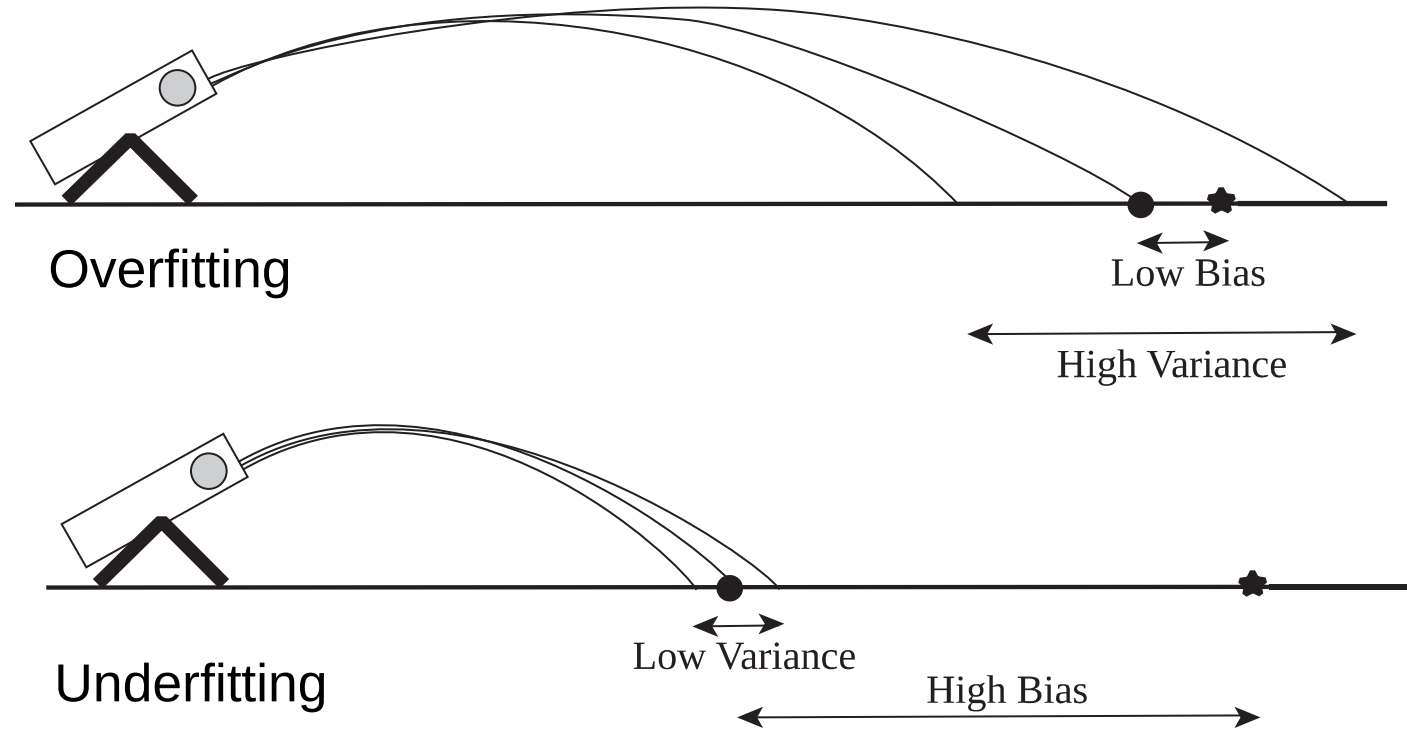
- Analogous problem of reaching a target y by firing projectiles from x (regression problem)



- For classification, gen. error of model m can be decomposed by:

$$gen.error(m) = c_1 \times noise + bias(m) + c_2 \times variance(m)$$

Fight the Bias-Variance trade-off



- Ensemble methods try to **reduce** the variance of complex models, but still **maintain** the low bias of simple models.

Bagging

- Decision trees
 - Pros: interpretable, can handle discrete and continuous features, robust to outliers, low bias, etc.
 - Cons: high variance
- Trees are perfect candidates for ensembles
 - Consider averaging many (nearly) unbiased tree estimators
 - Bias remains similar, but variance is reduced
- This is called bagging (bootstrap aggregating)
 - Train many trees on bootstrapped data, then take voting

Bagging

- Bootstrap sampling: sampling with replacement, same size as the original dataset (but with duplicate)

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

- Build classifier on each bootstrap sample
- Probability of a training instance being selected in a bootstrap sample is:
 - $1 - (1 - 1/n)^n$ (n: number of training instances)
 - ~ 0.632 when n is large

Random forest

- Reduce correlation between trees, by introducing randomness

1. For $b = 1, \dots, B$:

1. Draw a bootstrap dataset Z^*
2. Learn a tree $f_b(\cdot)$ on Z^* , in particular **select m features randomly** out of p features as candidates before splitting

2. Output:

- Regression: $f(x) = \frac{1}{B} \sum_{b=1}^B f_b(x)$
- Classification: majority vote

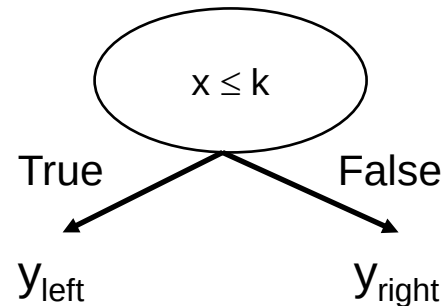
Bagging example

- Consider 1-dimensional data set:

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- Classifier is a decision stump (decision tree of size 1)
 - Decision rule: $x \leq k$ versus $x > k$
 - Split point k is chosen based on entropy



Bagging example

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging example

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

$x \leq 0.7 \rightarrow y = 1$

$x > 0.7 \rightarrow y = 1$

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.3 \rightarrow y = 1$

$x > 0.3 \rightarrow y = -1$

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging example

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

$x \leq 0.05 \rightarrow y = 1$

$x > 0.05 \rightarrow y = 1$

Bagging example

- Summary of Trained Decision Stumps:

Round	Split Point	Left Class	Right Class
1	0.35	1	-1
2	0.7	1	1
3	0.35	1	-1
4	0.3	1	-1
5	0.35	1	-1
6	0.75	-1	1
7	0.75	-1	1
8	0.75	-1	1
9	0.75	-1	1
10	0.05	1	1

Bagging example

- Use majority vote (sign of sum of predictions) to determine class of ensemble classifier

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Predicted Class Sign	1	1	1	-1	-1	-1	-1	1	1	1

- Bagging can increase the complexity (representation capacity) of simple classifiers such as decision stumps

Boosting

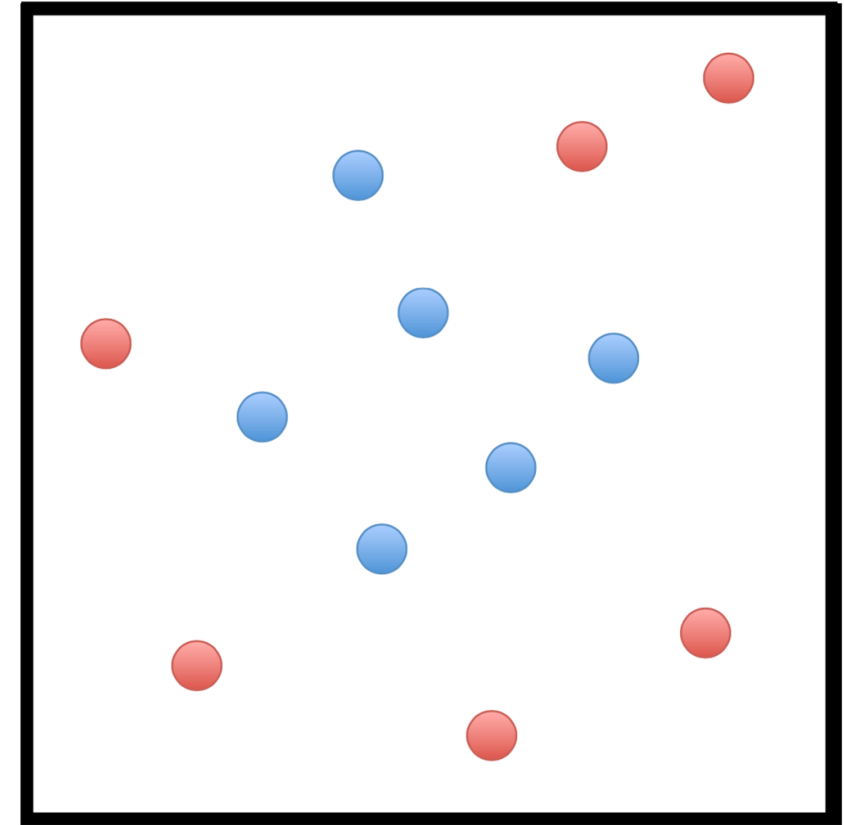
- An iterative procedure to adaptively change distribution of training data by focusing more on previously misclassified records
 - Initially, all N training instances are assigned equal weights
 - Wrongly classified will have their weights **increased** in the next round
 - Correctly classified will have their weights **decreased** in the next round
- On each iteration t
 - Weight each training instance by how incorrectly it was classified
 - Learn a weak model - h_t
 - A strength showing importance of model at current iteration - β_t
- Ensemble classifier

$$H(X) = \text{sign}\left(\sum \beta_t h_t(X)\right)$$

AdaBoost

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

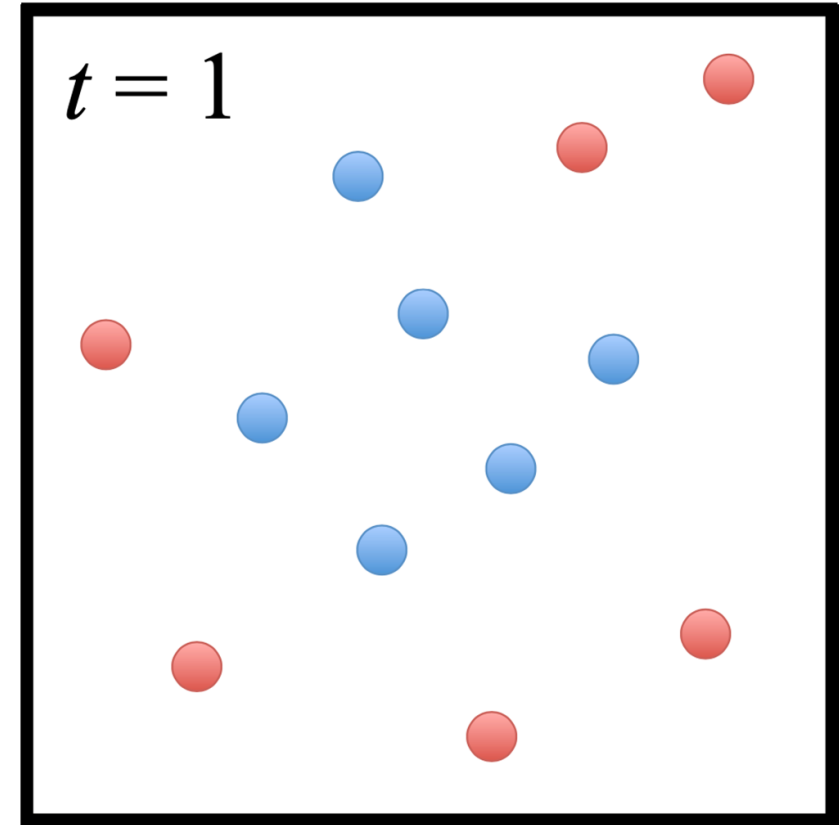


- Given the number of instance N , all have the size weights initially
- Size of point in figure represents the instance's weight

AdaBoost – iter 1

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

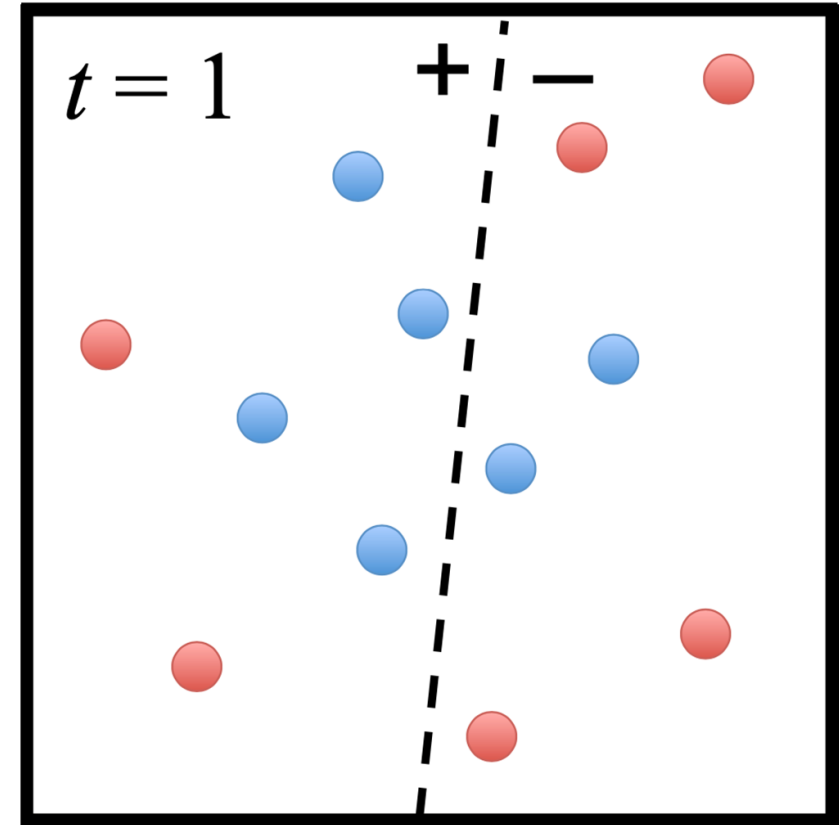


AdaBoost – iter 1

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- β_t measures the importance of model h_t
- If learner error $\epsilon_t \leq 0.5$ (random guess), then $\beta_t \geq 0$



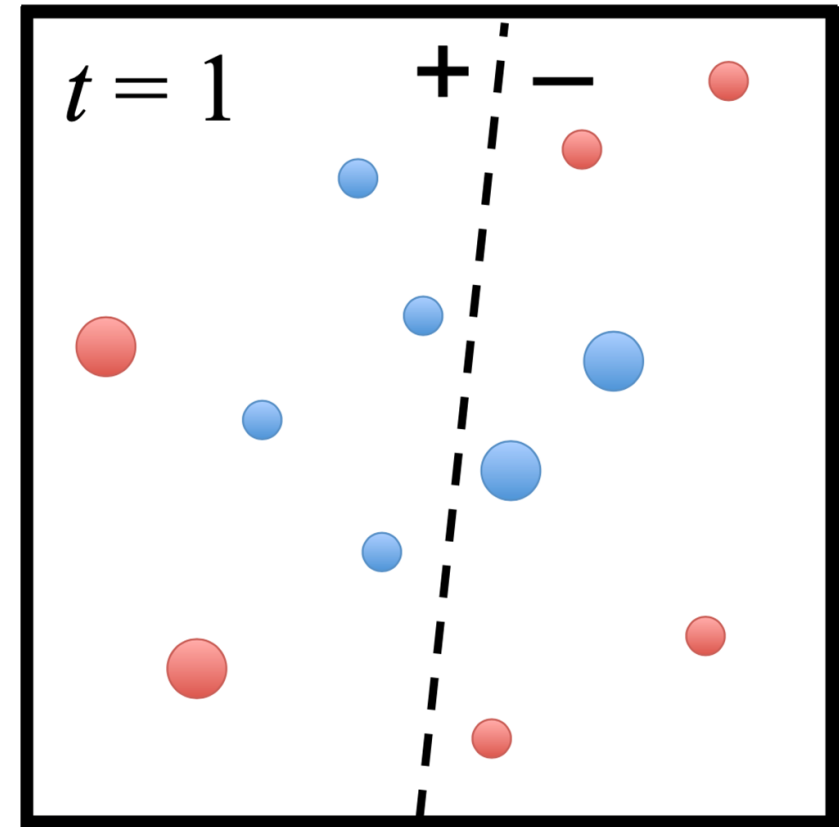
AdaBoost – iter 1

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Weights of correct predictions are multiplied by $e^{-\beta_t} \leq 1$
- Weights of **wrong** predictions are multiplied by $e^{\beta_t} \geq 1$

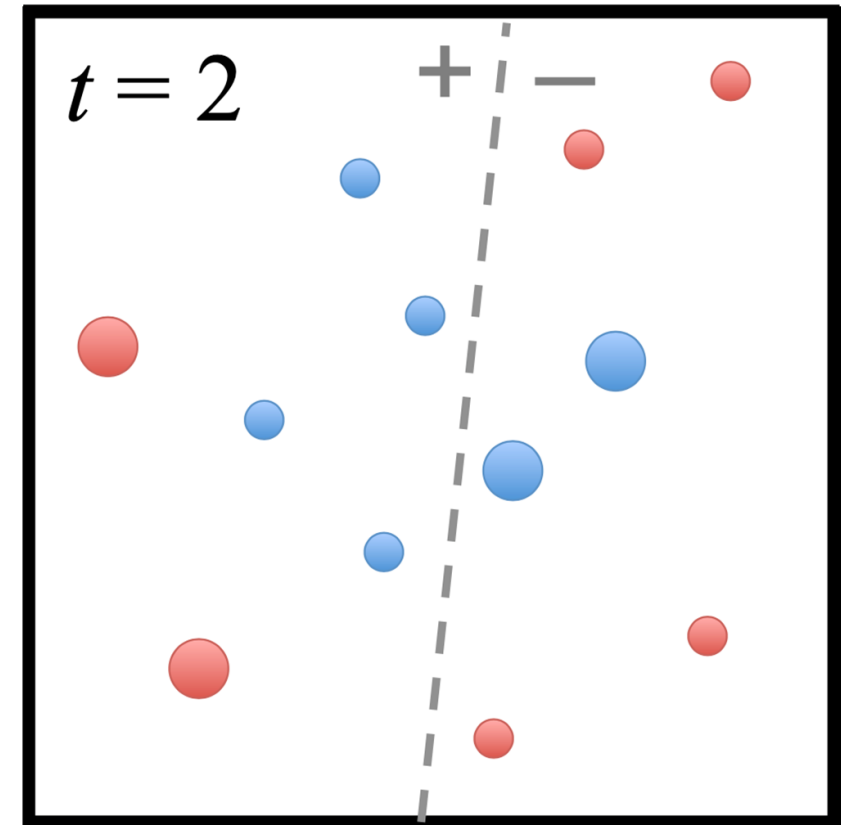
Weights update (by size of points)!



AdaBoost – iter 2

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

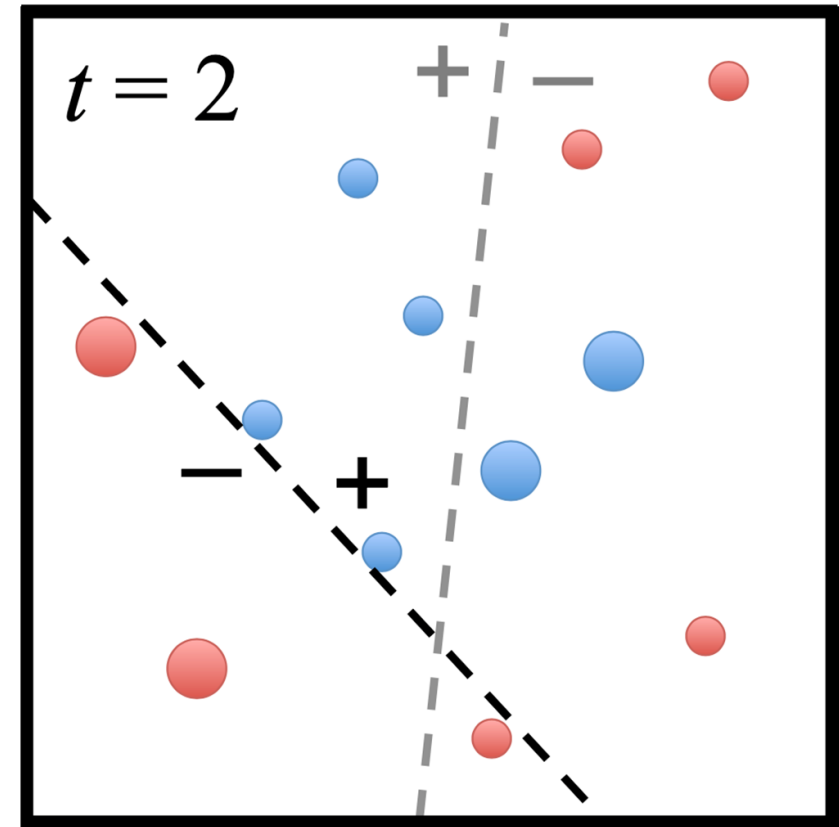


AdaBoost – iter 2

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- β_t measures the importance of model h_t
- If learner error $\epsilon_t \leq 0.5$ (random guess), then $\beta_t \geq 0$



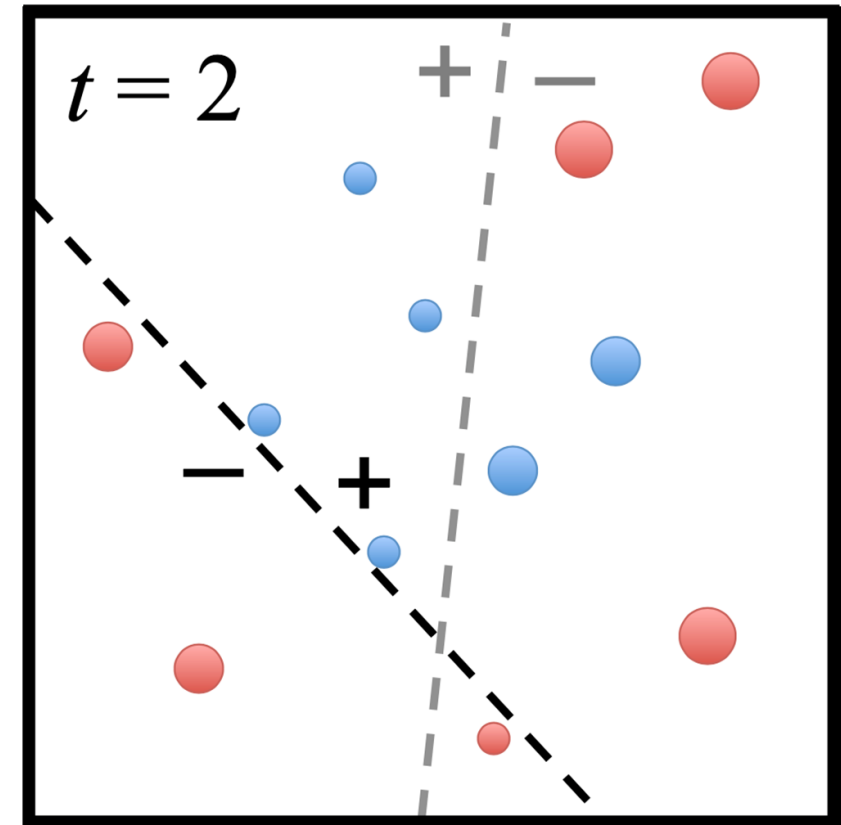
AdaBoost – iter 2

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Weights of correct predictions are multiplied by $e^{-\beta_t} \leq 1$
- Weights of **wrong** predictions are multiplied by $e^{\beta_t} \geq 1$

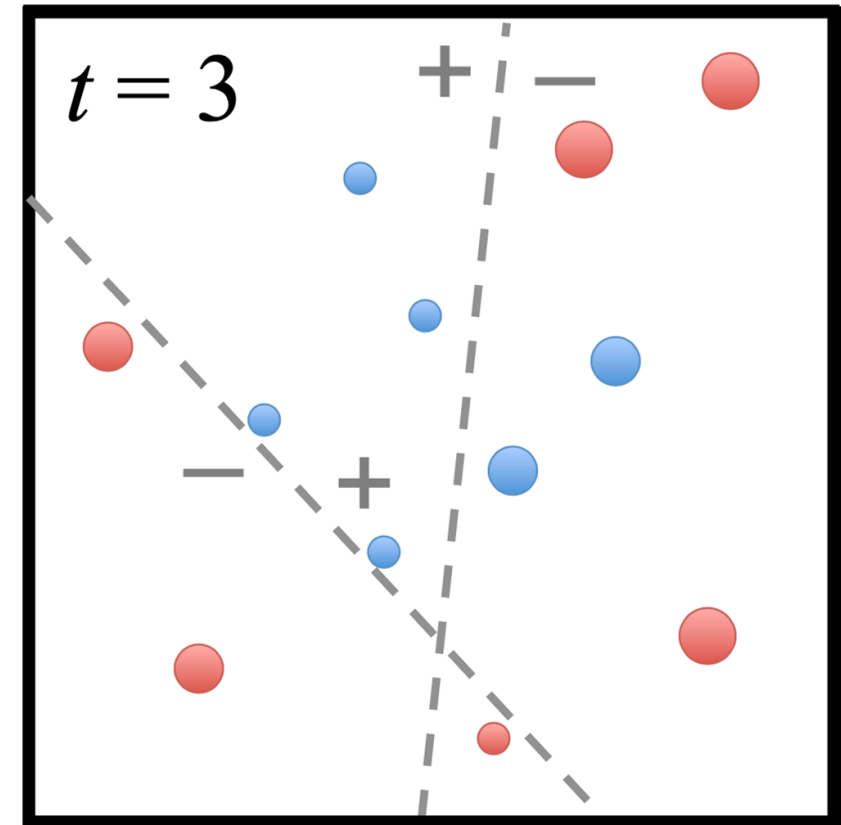
Weights update (by size of points)!



AdaBoost – iter 3

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

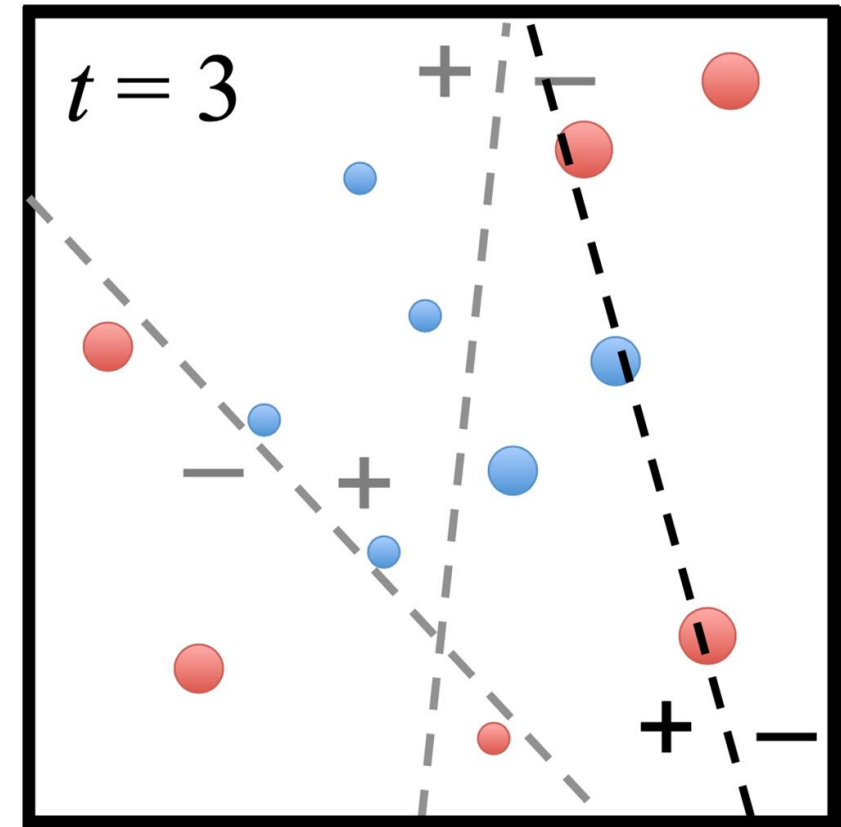


AdaBoost – iter 3

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- β_t measures the importance of model h_t
- If learner error $\epsilon_t \leq 0.5$ (random guess), then $\beta_t \geq 0$



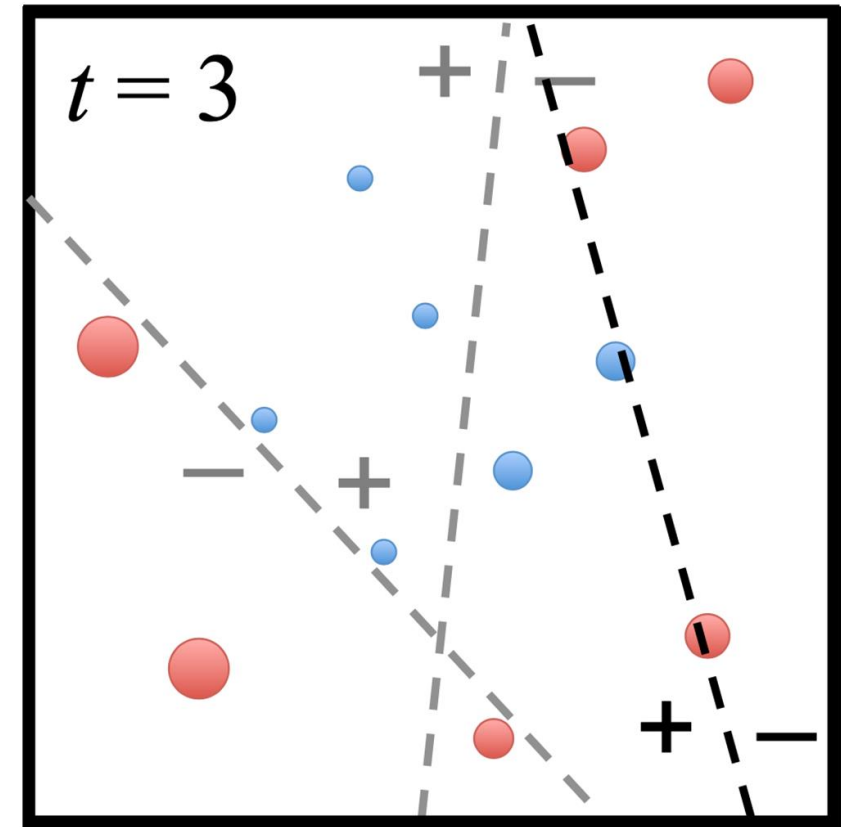
AdaBoost – iter 3

- 1: Initialize a vector of n uniform weights $\mathbf{w}_1$
- 2: **for** $t = 1, \dots, T$
- 3: Train model h_t on X, y with weights $\mathbf{w}_t$
- 4: Compute the weighted training error of h_t
- 5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Update all instance weights:
 $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i))$
- 7: Normalize $\mathbf{w}_{t+1}$ to be a distribution
- 8: **end for**
- 9: **Return** the hypothesis

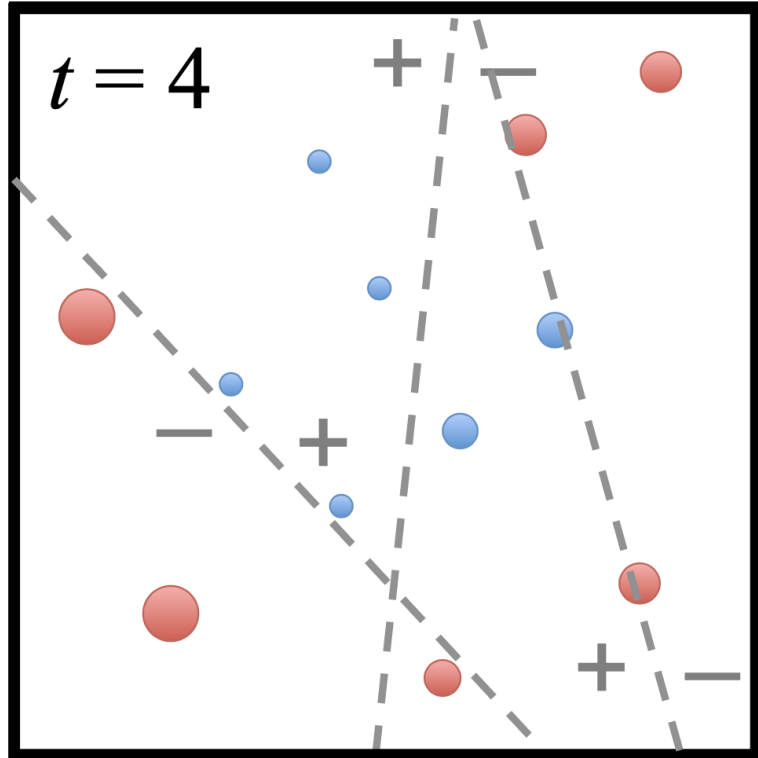
$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Weights of correct predictions are multiplied by $e^{-\beta_t} \leq 1$
- Weights of **wrong** predictions are multiplied by $e^{\beta_t} \geq 1$

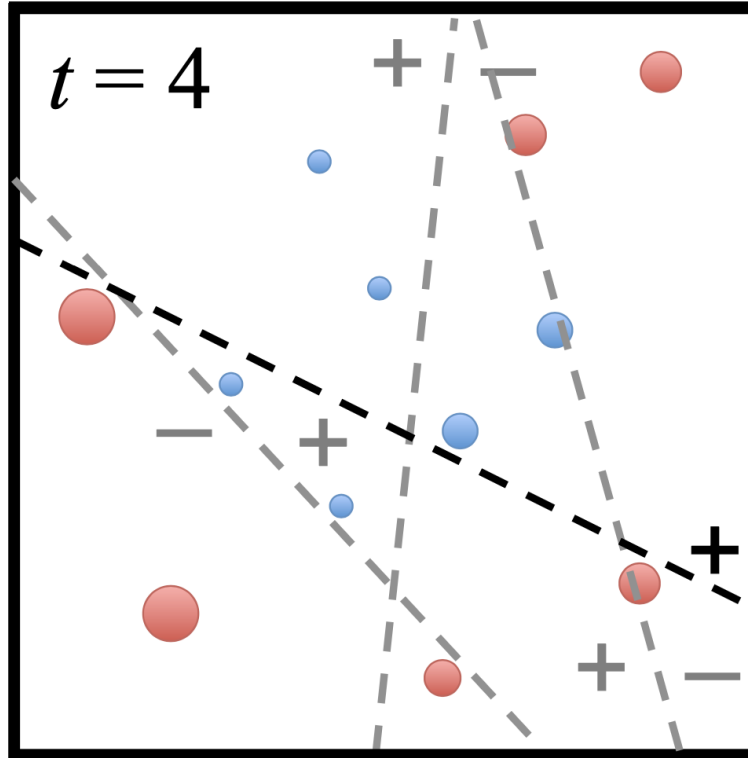
Weights update (by size of points)!



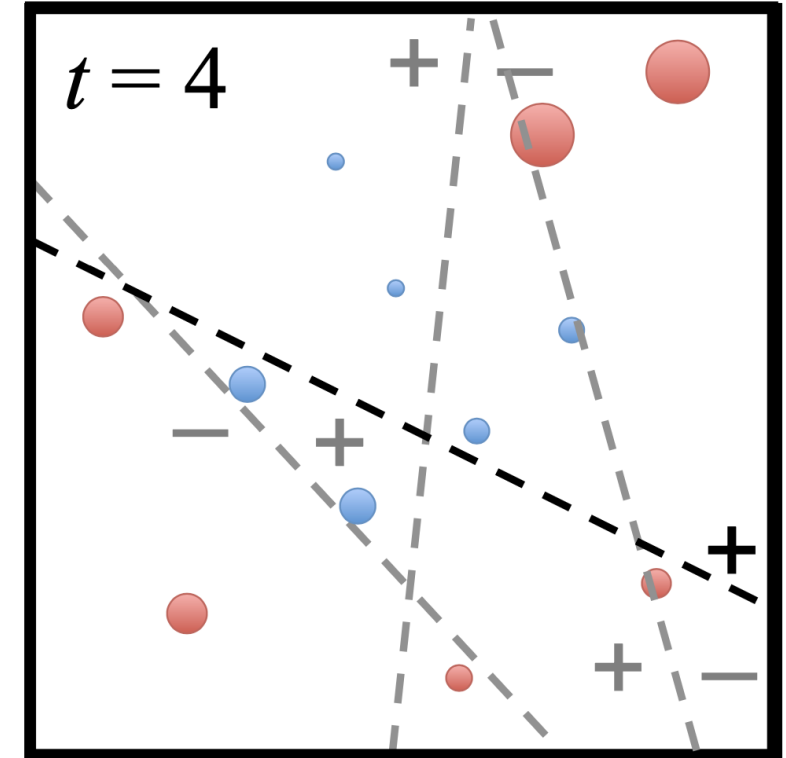
AdaBoost – iter 4



Train a model

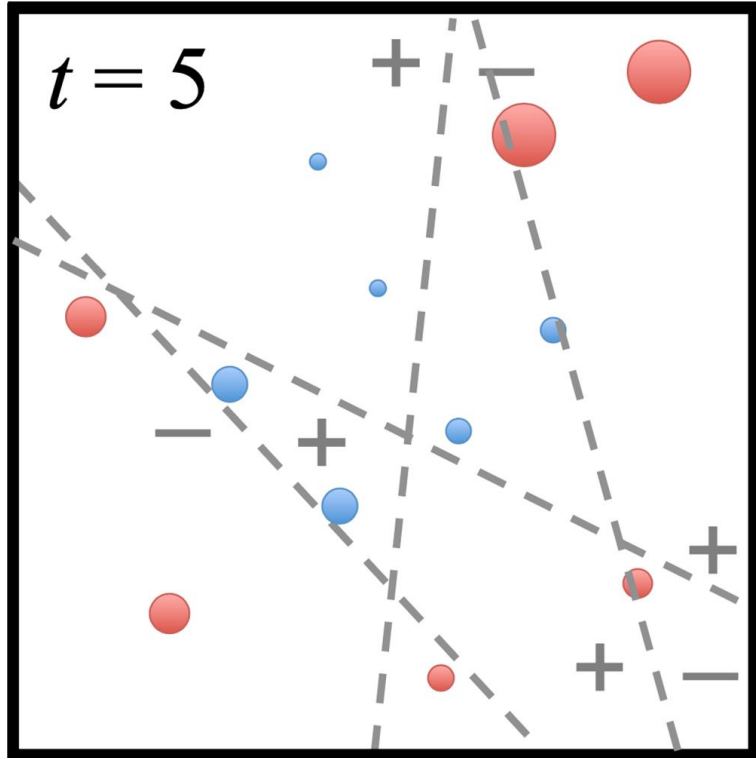


Calculate the error

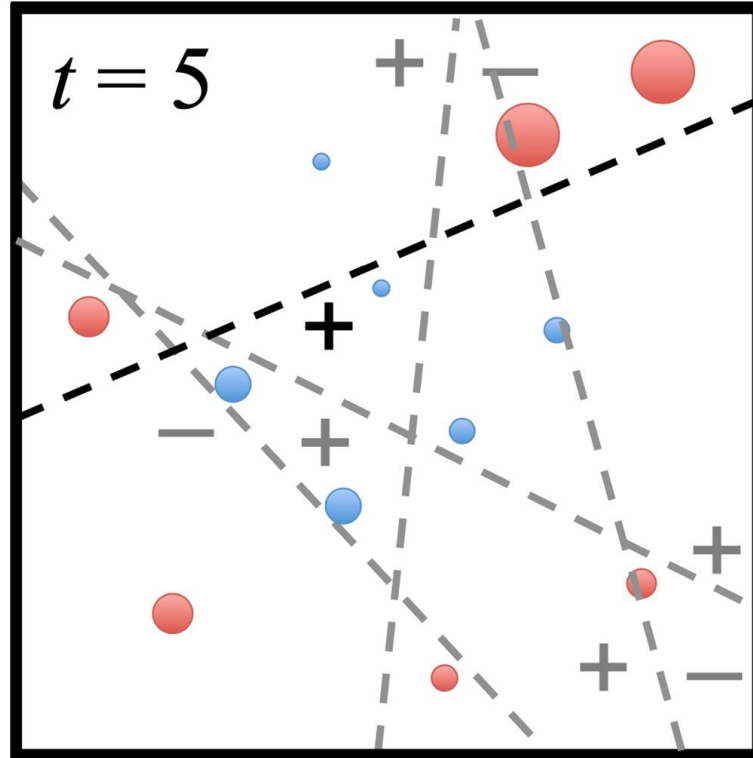


Weights update (by size of points)!

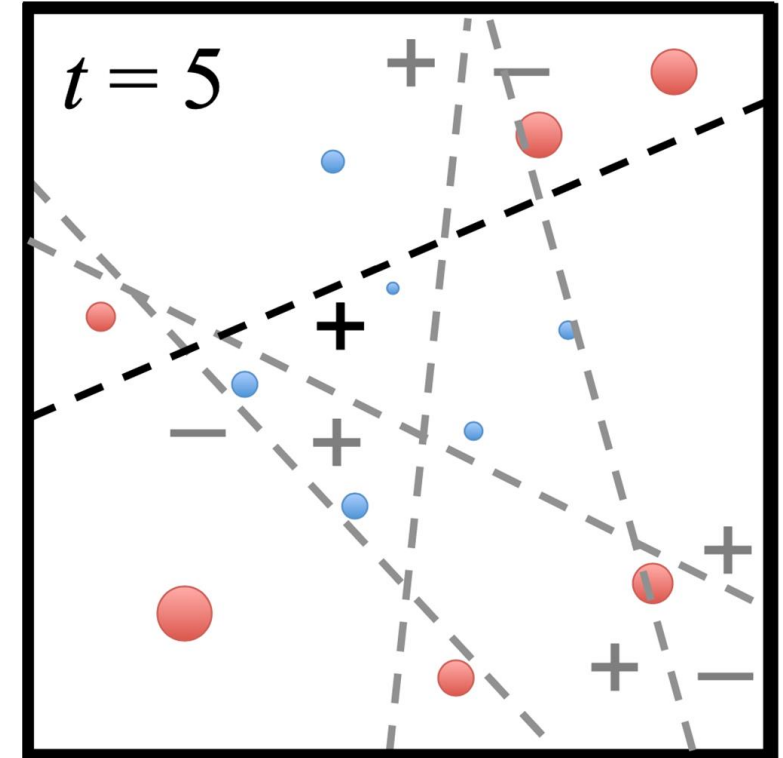
AdaBoost – iter 5



Train a model

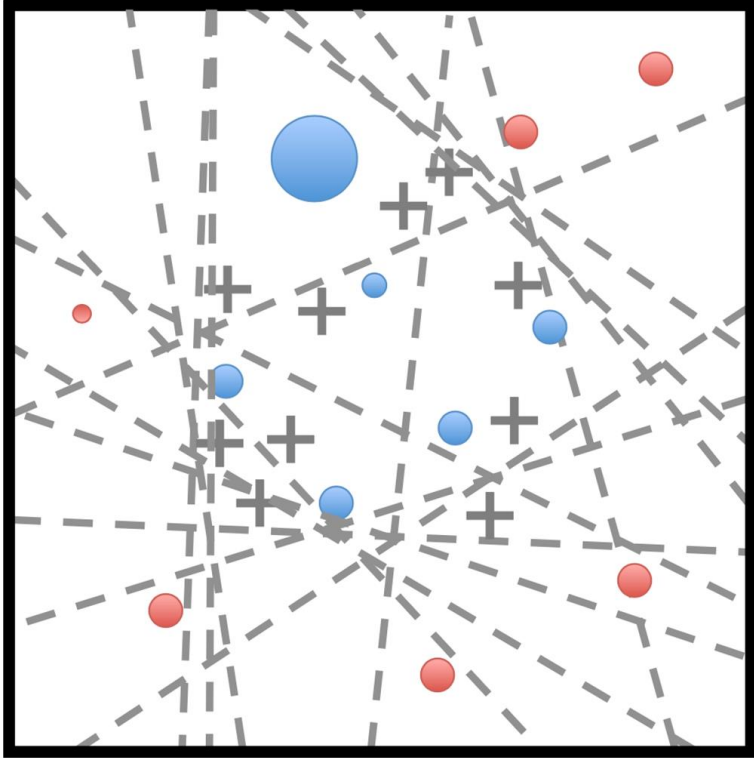


Calculate the error

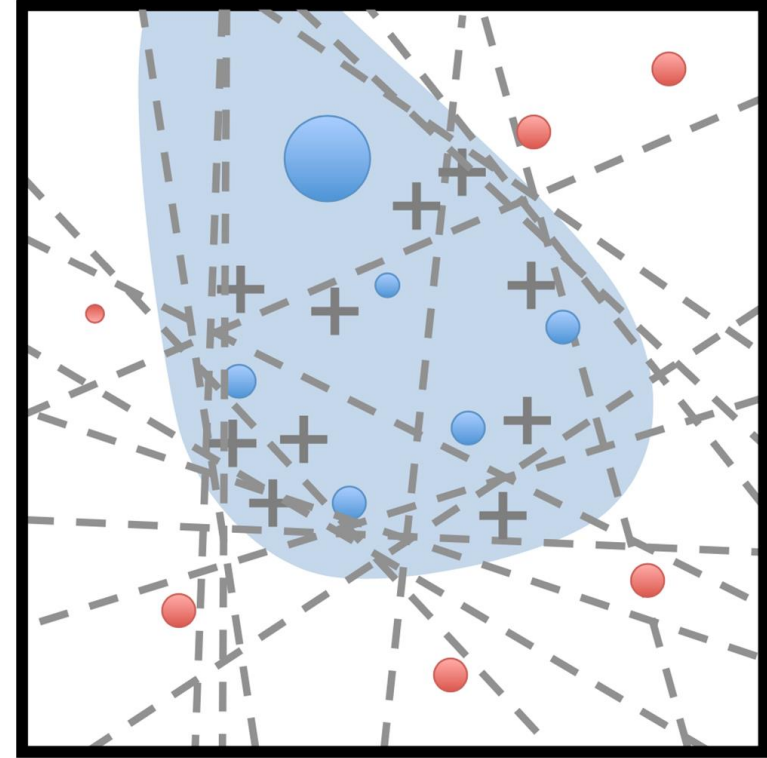


Weights update (by size of points)!

AdaBoost – many iterations

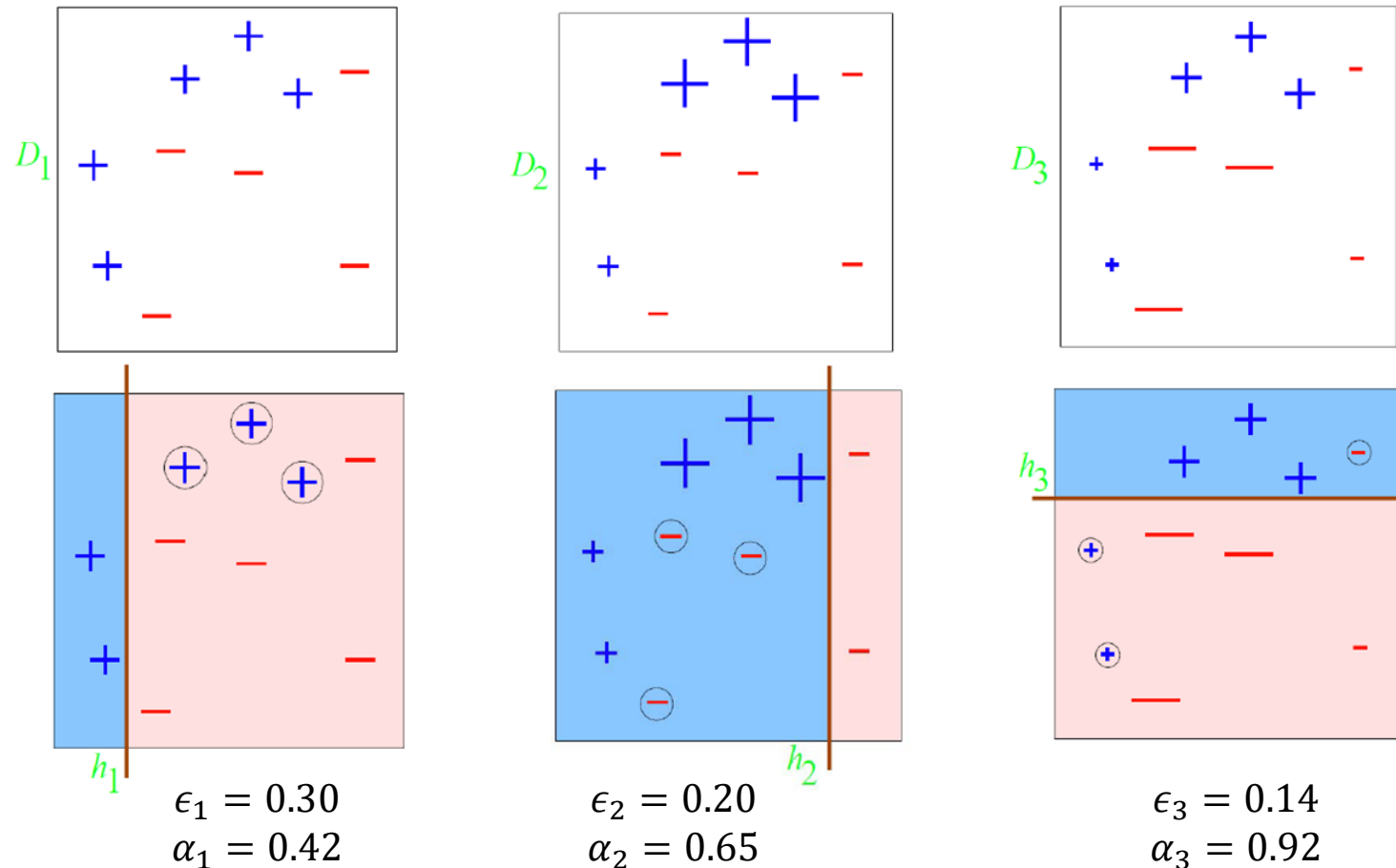


A lot of weak learners



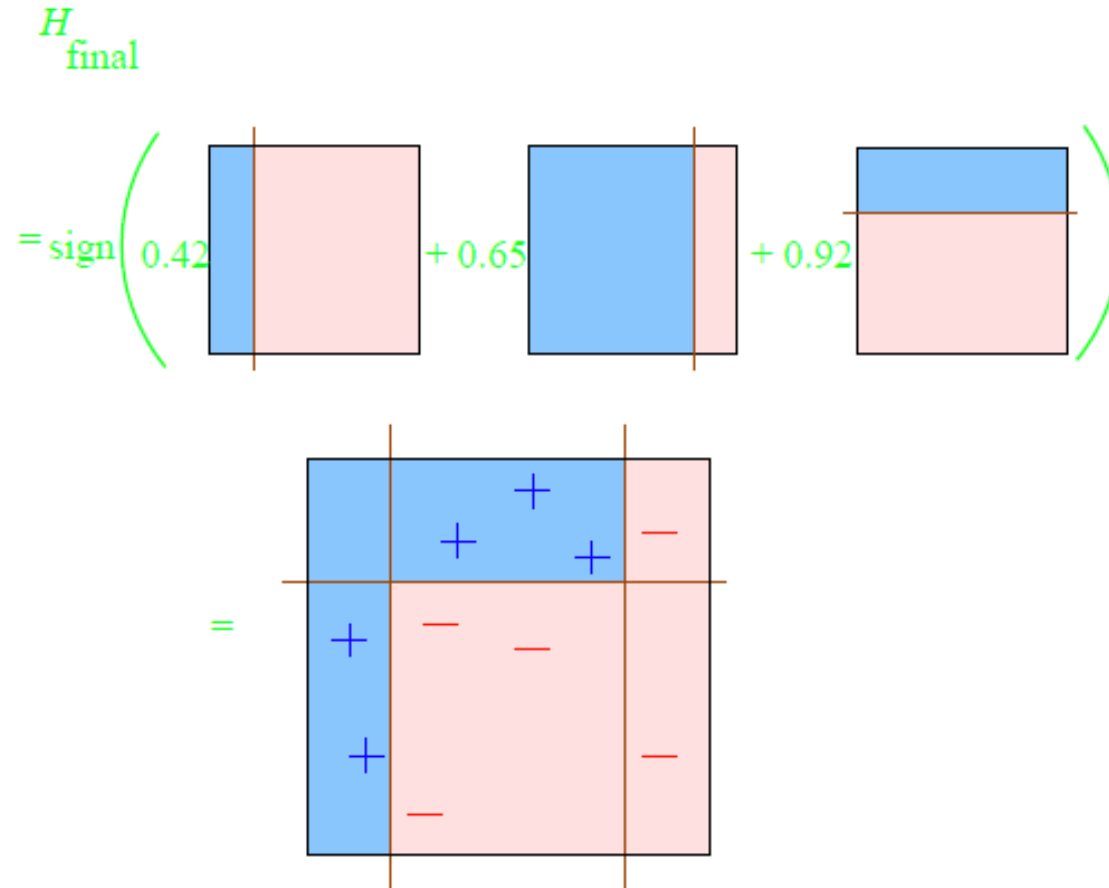
Combining all the members of by learner importance β . Final model can realize a complex decision boundary

Why does AdaBoost work



- Training points which are difficult to be classified will be given higher weight
- Learners which correctly classify difficult points will be given higher importance

Why does AdaBoost work



- The final decision are voting with the importance of each learner

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- $\mathbf{w}_t$ is a vector of weights over the training instances at iteration t
- All points start with equal weight

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- We need a way to weight instances differently when learning the model
- A simple way is to incorporate weights w into the cost function

$$J_{\text{reg}}(\boldsymbol{\theta}) = - \sum_{i=1}^n w_i [y_i \log h_{\boldsymbol{\theta}}(\mathbf{x}_i) + (1 - y_i) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}_i))]$$

Weights of $\mathbf{x}_i$ Cross-entropy loss of $\mathbf{x}_i$

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Error is the sum of the weights of all misclassified instances

- β_t measures the importance of h_t
- β_t grows as the error of h_t shrinks

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Emphasize the misclassified instances by increase their weights

This is the same as:

$$w_{t+1,i} = w_{t,i} \times \begin{cases} e^{-\beta_t} & \text{if } h_t(\mathbf{x}_i) = y_i \\ e^{\beta_t} & \text{if } h_t(\mathbf{x}_i) \neq y_i \end{cases}$$

will be ≤ 1
will be ≥ 1

- Make the weight for next iteration $\mathbf{w}_{t+1}$ sum up to 1

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Member classifiers with less error are given more weight in the final ensemble model
- Final prediction is a weighted combination of each member's prediction

Summary of AdaBoost

- Strengths
 - Fast and simple to program
 - No parameters to tune
 - No assumptions on weak learner
- When boosting can fail
 - Given insufficient data
 - Overly complex weak model
 - Can be susceptible to noise
 - When there are a large number of outliers